Lab Report: MicroPython 1 - Lab 1

Embedded Electronic Systems

Submitted by: Yosi Brim, Elad Asbag
Instructor: Dr. Yosef Golovcov

March 15, 2026

Introduction

This report presents the solutions for the MicroPython lab using an ESP32 microcontroller in the Wokwi simulation environment. To provide a clean and robust solution, all four required tasks were combined into a single, menu-driven Python program. The user is prompted to select a task number via the serial terminal, after which the specific functionality is executed.

The report includes the unified source code followed by the expected results and simulation proofs for each of the four menu options.

1 Unified Source Code

The following MicroPython script configures the necessary GPIO and PWM pins and runs a continuous loop, allowing the user to select and execute any of the four lab tasks.

```python
print("Hello, ESP32!")
import machine
import time

# --- Pin Configuration (Setup) ---

# Task 1: Blinking LED
led_task1 = machine.Pin(2, machine.Pin.OUT)

# Task 2: 4 LEDs
leds_task2 = [machine.Pin(i, machine.Pin.OUT) for i in (12, 13, 14, 15)]

# Task 3: Separate colored LEDs
led_red = machine.Pin(16, machine.Pin.OUT)
led_green = machine.Pin(17, machine.Pin.OUT)
led_blue = machine.Pin(18, machine.Pin.OUT)

# Task 4: RGB LED (Using PWM to allow creating shades like orange and purple)
pwm_r = machine.PWM(machine.Pin(25), freq=1000)
pwm_g = machine.PWM(machine.Pin(26), freq=1000)
pwm_b = machine.PWM(machine.Pin(27), freq=1000)

def set_rgb_color(r, g, b):
```

```python
    # In ESP32, PWM duty values range from 0 to 1023
    # This function converts standard RGB values (0-255) to PWM values
    pwm_r.duty(int((r / 255) * 1023))
    pwm_g.duty(int((g / 255) * 1023))
    pwm_b.duty(int((b / 255) * 1023))

# --- Task Functions ---

def task1():
    try:
        num = int(input("Enter number of LED blinks: "))
        for _ in range(num):
            led_task1.value(1)
            time.sleep(0.5)
            led_task1.value(0)
            time.sleep(0.5)
    except ValueError:
        print("Error: Please enter an integer.")

def task2():
    try:
        num = int(input("Enter a number between 1 and 4: "))
        if 1 <= num <= 4:
            for i in range(4):
                # Turns on LEDs up to the input number, turns off the rest
                leds_task2[i].value(1 if i < num else 0)
        else:
            print("Number out of range!")
    except ValueError:
        print("Error: Please enter an integer.")

def task3():
    color = input("Enter color (red, green, blue): ").strip().upper()

    # Turn off all LEDs before turning one on
    led_red.value(0)
    led_green.value(0)
    led_blue.value(0)

    if color == "RED":
        led_red.value(1)
    elif color == "GREEN":
        led_green.value(1)
    elif color == "BLUE":
        led_blue.value(1)
    else:
        print("Unknown color.")

```

```python
def task4():
    color = input("Enter color (red, green, blue, purple, orange, yellow, white): ").strip().upper()

    # Dictionary containing RGB values for each color
    colors = {
        "RED": (255, 0, 0),
        "GREEN": (0, 255, 0),
        "BLUE": (0, 0, 255),
        "PURPLE": (128, 0, 128),
        "ORANGE": (255, 165, 0),
        "YELLOW": (255, 255, 0),
        "WHITE": (255, 255, 255)
    }

    if color in colors:
        set_rgb_color(*colors[color])
    else:
        print("Unknown color.")
        set_rgb_color(0, 0, 0) # Turn off in case of error

# --- Main Loop ---

while True:
    print("\n--- Task Menu ---")
    print("1. Blink LED by number")
    print("2. Turn on amount of LEDs (1-4)")
    print("3. Turn on separate LED by color")
    print("4. Turn on RGB LED by color")
    print("5. Exit")

    choice = input("Select a task (1-5): ")

    if choice == '1':
        task1()
    elif choice == '2':
        task2()
    elif choice == '3':
        task3()
    elif choice == '4':
        task4()
    elif choice == '5':
        print("Exiting program.")
        break
    else:
        print("Invalid choice, please try again.")
```

2 Execution and Results

This section demonstrates the system's response to each of the four menu options.

2.1 Menu Option 1: Number Input and LED Blinking

Expected Results: Upon selecting option 1 and entering an integer N, GPIO pin 2 alternates between logic HIGH and LOW states exactly N times, with a 500ms delay between changes, causing the LED to blink.

Simulation Proof: The serial terminal captures the user selecting task '1' and inputting the blink count. The schematic demonstrates the ESP32 successfully driving the LED.

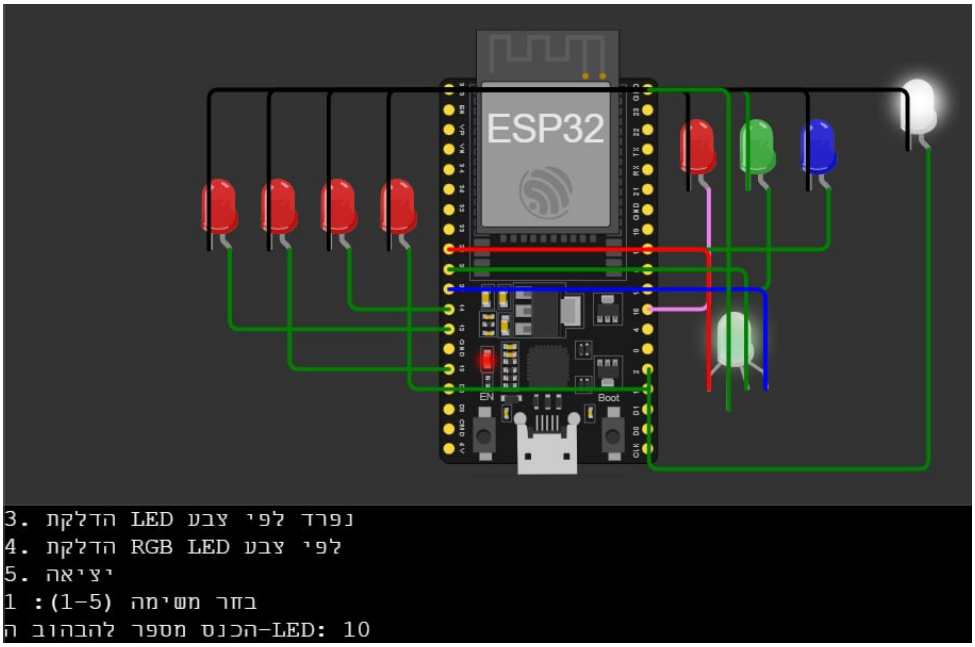

Figure 1: Execution of Menu Option 1: Blinking the LED.

2.2 Menu Option 2: Lighting LEDs Based on Input (1-4)

Expected Results: After selecting option 2 and inputting an integer N ($1 \leq N \leq 4$), the microcontroller drives exactly N consecutive GPIO pins (starting from pin 12) to a logic HIGH state, while ensuring the rest in the array are driven to LOW.

Simulation Proof: The terminal shows the selection of task '2' and a valid numerical input. The schematic verifies that the software maps the input to the physical output pins correctly, lighting the exact requested amount of LEDs.

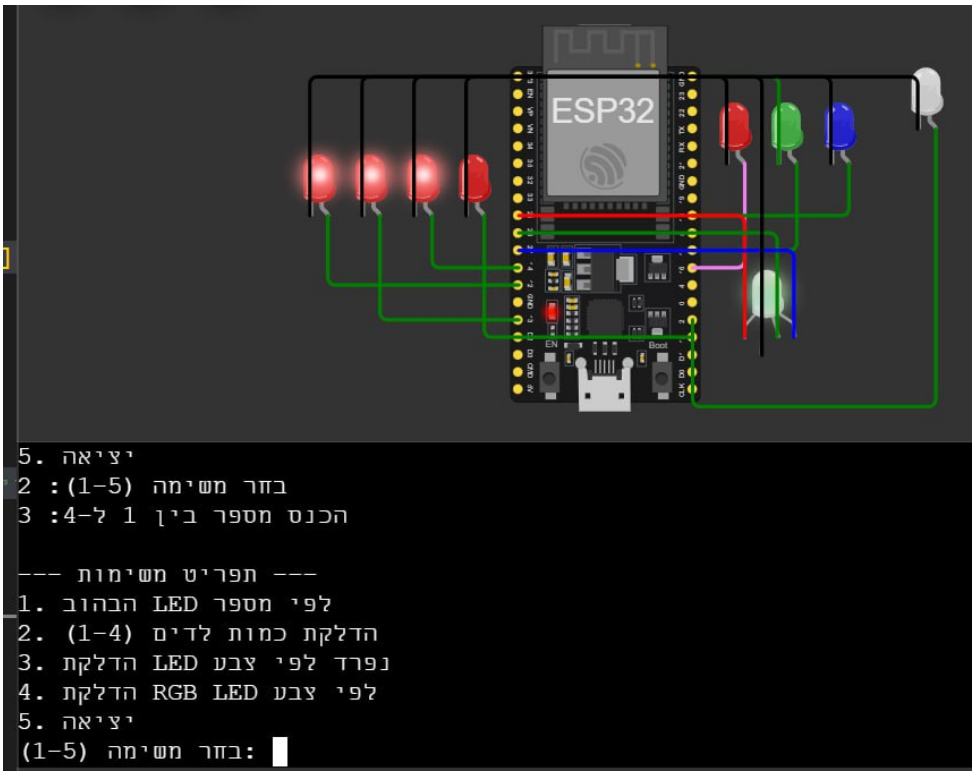

Figure 2: Execution of Menu Option 2: Lighting the specific number of LEDs.

2.3 Menu Option 3: Lighting the Corresponding Color LED

Expected Results: Upon choosing option 3 and entering a color string, conditional logic drives the specific GPIO pin associated with that color (pins 16, 17, or 18) to HIGH, actively driving the other color pins to LOW.

Simulation Proof: The terminal captures the selection of task '3' and the color input. The schematic confirms that only the corresponding discrete LED receives voltage.

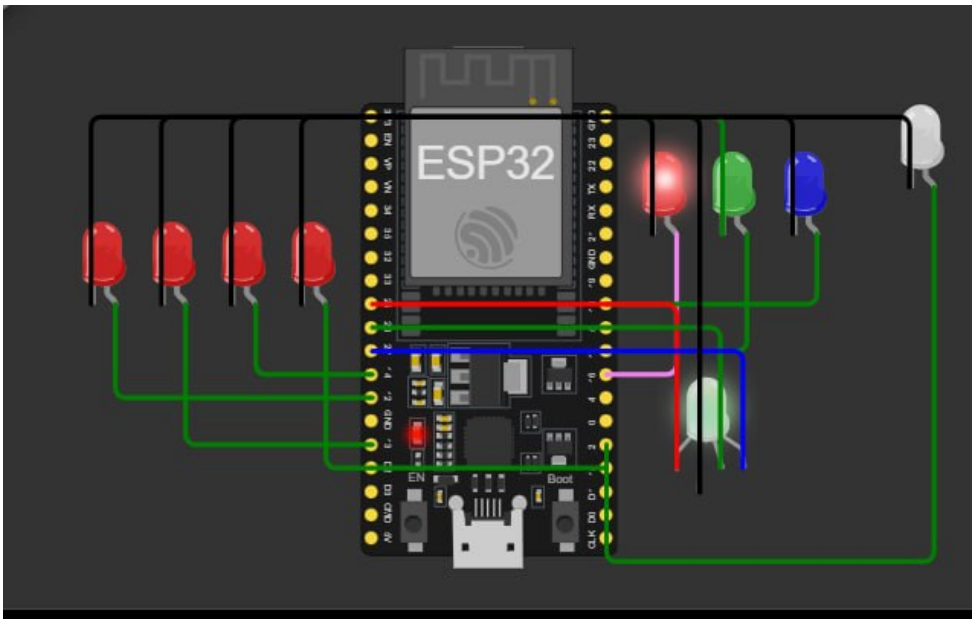

Figure 3: Execution of Menu Option 3: Turning on the specific matching LED.

2.4 Menu Option 4: Using an RGB LED with PWM

Expected Results: Selecting option 4 and entering a complex color string maps the choice to an 8-bit RGB value. This is scaled to a 10-bit PWM duty cycle (0-1023). The microcontroller outputs specific PWM signals on pins 25, 26, and 27 to visually mix the requested color inside the RGB component.

Simulation Proof: The terminal shows the selection of task '4' and a complex color input. The simulation demonstrates the RGB LED emitting the correct wavelength based on the calculated PWM duty cycles.

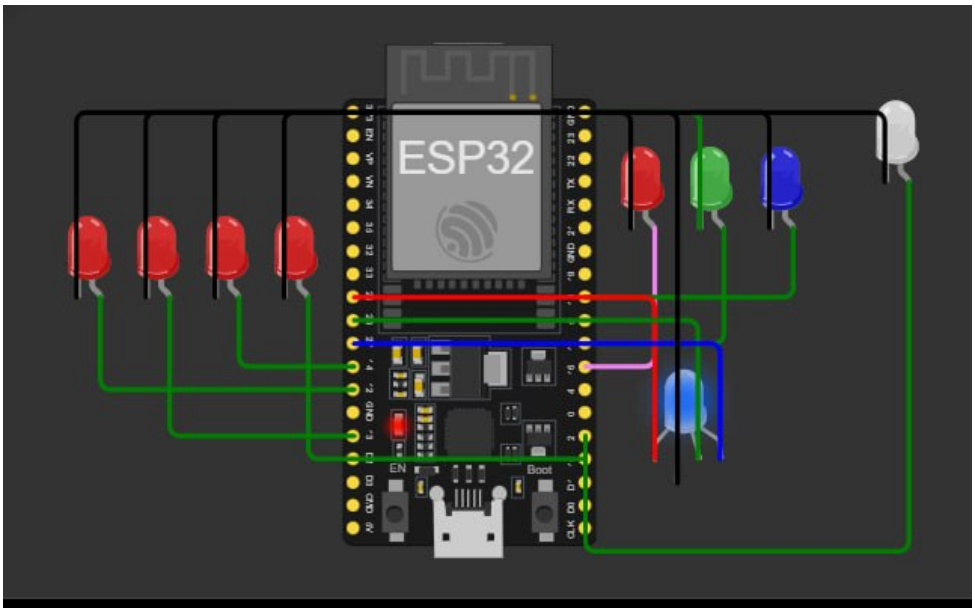

Figure 4: Execution of Menu Option 4: Generating mixed colors using PWM.

3 Conclusion

In conclusion, this laboratory exercise successfully demonstrated the fundamental principles of interfacing an ESP32 microcontroller with external hardware components using MicroPython. By implementing a unified, menu-driven program, we explored basic digital output control (GPIO) for driving discrete LEDs and utilized advanced Pulse Width Modulation (PWM) techniques for precise color mixing with an RGB LED. The Wokwi simulation environment proved to be an effective platform for validating our hardware logic and software design prior to physical implementation. Overall, these tasks provided practical, hands-on experience in developing robust embedded systems, reinforcing the theoretical concepts of microcontroller-based control.